

Aim: Write a user defined function in python to

- a) **SHIFT(lst)** that would accept a list as argument .The function should shift the negative numbers of the list to right and the positive numbers to left without using a another list.

For example if list initially contains

[3, -5, 1, 3, 7, 0, -15, 3, -7, -8]

Then after shifting list should contain

[3, 1, 3, 7, 0, 3, -8, -7, -15, -5]

Source Code:

```
def SHIFT(lst):
    left = 0
    right = len(lst) - 1
    while left <= right:
        if lst[left] >= 0:
            left += 1
        elif lst[right] < 0:
            right -= 1
        else:
            lst[left], lst[right] = lst[right], lst[left]
            left += 1
            right -= 1
    print(lst)
lst=eval(input("Enter the list:"))
SHIFT(lst)
```

Output:

Enter the list:[1,-1,-2,2,9,0,1]

[1, 1, 0, 2, 9, -2, -1]

- b) **SwapHalfList(Array)**, which accepts a list Array of numbers and swaps the elements of 1st Half of the list with the 2nd Half of the list ONLY if the sum of 1st Half is greater than 2nd Half of the list.

Sample Input Data of the list:

Array= [100, 200, 300, 40, 50, 60]

Output

Arr = [40, 50, 60, 100, 200, 300]

Source Code:

```
def SwapHalfList(Array):
    mid = len(Array) // 2
    if sum(Array[:mid]) > sum(Array[mid:]):
        for i in range(mid):
            Array[i], Array[mid + i] = Array[mid + i], Array[i]
    print(Array)
Array=eval(input("Enter the list:"))
SwapHalfList(Array)
```

Output:

Enter the list:[10,3,4,1,2,3]

[1, 2, 3, 10, 3, 4]

- c) **RShift(Lst)** in Python, which accepts a list Lst of numbers shifts all its elements right once.

Sample Input Data of the list:

Lst= [10,20,30,40,12,11]

Output:

Lst = [11,10,20,30,40,12]

Source Code:

```
def RShift(Lst):  
    last = Lst[-1]  
    for i in range(len(Lst) - 1, 0, -1):  
        Lst[i] = Lst[i - 1]  
    Lst[0] = last  
    print(Lst)
```

```
lst=eval(input("Enter the list:"))  
RShift(lst)
```

Output:

```
Enter the list:[1, 1, 0, 2, 9, -2, -1]  
[-1, 1, 1, 0, 2, 9, -2]
```

- d) **LShift(Lst,n)** in Python, which accepts a list Lst of numbers and n is a numeric value by which all elements of the list are shifted to left.

Sample Input Data of the list:

Lst= [10,20,30,40,12,11], n=2

Output:

Lst = [30,40,12,11,10,20]

Source Code:

```
def LShift(Lst, n):  
    for k in range(n):  
        first = Lst[0]  
        for i in range(len(Lst) - 1):  
            Lst[i] = Lst[i + 1]  
        Lst[-1] = first  
    print(Lst)
```

```
Lst=eval(input("Enter the list:"))  
n=int(input("Enter the number of Shift:"))  
LShift(Lst, n)
```

Output:

```
Enter the list:[0,1,2,3,4,5]
```

```
Enter the number of Shift:3
```

```
[3, 4, 5, 0, 1, 2]
```

- e) **lst_reverse()** that reverses a list. Write the code without slicing/built-in function.

Sample list: [12,34,5,67,43] After execution of the code: [43, 67, 5, 34, 12]

Source Code:

```
# 1(e)
def lst_reverse(lst):
    i = 0
    j = len(lst) - 1
    while i < j:
        lst[i], lst[j] = lst[j], lst[i]
        i += 1
        j -= 1
    print(lst)
lst=eval(input("Enter the list:"))
lst_reverse(lst)
```

Output:

Enter the list:[3, 4, 5, 0, 1, 2]
[2, 1, 0, 5, 4, 3]

- f) **part_reverse(lst,start, end)** to reverse elements in a list where arguments are start and end index of the list part which is to be reversed.

Assume that, $start < end$, $start \geq 0$ and $end < len(list)$

Sample Input Data of List:

my_list=[1,2,3,4,5,6,7,8,9,10]

Function Call = part_reverse(my_list,3,6)

Output is

my_list=[1,2,3,7,6,5,4,8,9,10]

Source Code:

```
def part_reverse(lst, start, end):
    while start < end:
        lst[start], lst[end] = lst[end], lst[start]
        start += 1
        end -= 1
    print(lst)
lst=eval(input("Enter the list:"))
start=int(input("Enter the start value:"))
end=int(input("Enter the stop value:"))
part_reverse(lst, start, end)
```

Output:

Enter the list:[1,2,3,4,5,6]

Enter the start value:2

Enter the stop value:5

[1, 2, 6, 5, 4, 3]

- g) **lst_modify()** to read a list of elements. Modify this list so that it does not contain any duplicate elements i.e. all elements occurring multiple times in the list should appear only once.

Source Code:

```
def lst_modify():
    lst = eval(input("Enter list: "))
    i = 0
    while i < len(lst):
        j = i + 1
        while j < len(lst):
            if lst[i] == lst[j]:
                lst.pop(j)
            else:
                j += 1
        i += 1
    print(lst)
lst_modify()
```

Output:

Enter list: [1,2,3,1,3,5,6]
[1, 2, 3, 5, 6]

h) **swap_loc()** to swap elements at the even location with the elements at the odd location.

Sample Input Data of the list:

Array= [100, 200, 300, 40, 50, 60]

Output

Array=[200,100,40,300,60,50]

Source Code:

```
def swap_loc(Array):
    for i in range(0, len(Array) - 1, 2):
        Array[i], Array[i + 1] = Array[i + 1], Array[i]
    print(Array)
Array=eval(input("Enter the list:"))
swap_loc(Array)
```

Output:

Enter the list:[1,2,3,4,5,6,7]
[2, 1, 4, 3, 6, 5, 7]

Exp No: 2

Date: dd-mm-yyyy

Aim: Consider the following student data:

```
data = [ ["Alice", 101, [85, 78, 92, 89]], ["Bob", 102, [76, 85, 90, 80]],
         ["Charlie", 103, [92, 88, 94, 91]], ["David", 104, [60, 72, 70, 65]],.....]
```

Write Python user defined functions to:

- Calculate and display each student's average marks.
- Identify and display the students who have scored below 70 in any subject.

Source Code:

```

data = [
    ["Alice", 101, [85, 78, 92, 89]],
    ["Bob", 102, [76, 85, 90, 80]],
    ["Charlie", 103, [92, 88, 94, 91]],
    ["David", 104, [60, 72, 70, 65]]
]

def display_average(data):
    print("Average Marks of Students")
    for student in data:
        avg = sum(student[2]) / len(student[2])
        print(student[0], "Roll No:", student[1], "Average:", avg)

def below_70(data):
    print("\nStudents who scored below 70 in any subject")
    for student in data:
        for mark in student[2]:
            if mark < 70:
                print(student[0], "Roll No:", student[1], "Marks:", student[2])
                break

display_average(data)
below_70(data)

```

Output:

```

Average Marks of Students
Alice Roll No: 101 Average: 86.0
Bob Roll No: 102 Average: 82.75
Charlie Roll No: 103 Average: 91.25
David Roll No: 104 Average: 66.75

```

```

Students who scored below 70 in any subject
David Roll No: 104 Marks: [60, 72, 70, 65]

```

Exp No: 3

Date: dd-mm-yyyy

Aim: Write a user defined function that simulates a dice roll. The program should generate and display a random number between 1 and 6 (inclusive) each time it is run. Use the random module. Run the program 15 times and count and display how many times each number (1–6) appears.

Source Code:

```

import random

def dice_roll():
    return random.randint(1, 6)

count = {1:0,2:0,3:0,4:0,5:0,6:0}
for i in range(15):
    n = dice_roll()
    print(n, end=' ')
    count[n] += 1
print('\nFrequency:')
for i in count:
    print(i,":",count[i])

```

Output:

3 6 5 5 4 2 5 4 3 2 2 3 5 1 2

Frequency:

1 : 1
2 : 4
3 : 3
4 : 2
5 : 4
6 : 1

Exp No:4**Date: dd-mm-yyyy****Aim:** Consider the following:

```
emp = {"John": {"January": 4500, "February": 5200, "March": 4800 },  
       "Emily": {"January": 6000, "February": 6200, "March": 5800 },.....}
```

Write Python user defined functions to:

- Increase the sales by 10% for employees whose average sales exceed 6000.
- Identify the month in which each employee had their highest sales(display name & month).
- Display the details of those employees whose name begin with a vowel.

Source Code:

```
emp = {'John': {'January':4500, 'February':5200, 'March':4800},  
       'Emily': {'January':6000, 'February':6200, 'March':5800},  
       'Anu': {'January':7000, 'February':7500, 'March':7200}}  
  
def increase_sales():  
    for name in emp:  
        avg = sum(emp[name].values()) / len(emp[name])  
        if avg > 6000:  
            for m in emp[name]:  
                emp[name][m] = emp[name][m] * 1.10  
    print(emp)  
    print()  
  
def highest_month():  
    for name, sales in emp.items():  
        highest = 0  
        highest_month_name = ""  
  
        for month, amount in sales.items():  
            if amount > highest:  
                highest = amount  
                highest_month_name = month  
  
        print(name, "had highest sales in", highest_month_name, "with sales", highest)  
        print()  
def vowel_names():  
    for name in emp:  
        if name[0].lower() in 'aeiou':  
            print(name, emp[name])  
increase_sales()  
highest_month()  
vowel_names()
```

Output:

```
{'John': {'January': 4500, 'February': 5200, 'March': 4800}, 'Emily': {'January': 6000, 'February': 6200, 'March': 5800}, 'Anu': {'January': 7700.000000000001, 'February': 8250.0, 'March': 7920.000000000001}}
```

John had highest sales in February with sales 5200

Emily had highest sales in February with sales 6200

Anu had highest sales in February with sales 8250.0

Emily {'January': 6000, 'February': 6200, 'March': 5800}

Anu {'January': 7700.000000000001, 'February': 8250.0, 'March': 7920.000000000001}

Exp No: 5

Date: dd-mm-yyyy

Aim: Consider the following data format:

```
{101:['Andrew',78],102:['Mark',99],103:['John',100],...}
```

Write user defined functions to:

- Create a dictionary with the roll number, name and marks of n students in a class.
- display the names of students who have marks above 75.

Source Code:

```
d = {}
def create_dict():
    n = int(input('Number of students: '))
    for i in range(n):
        roll = int(input('Roll no: '))
        name = input('Name: ')
        marks = int(input('Marks: '))
        d[roll] = [name, marks]
    return d

def above_75(d):
    for roll in d:
        if d[roll][1] > 75:
            print(d[roll][0])
create_dict()
above_75(d)
```

Output:

Exp No: 6

Date: dd-mm-yyyy

Aim: Consider the table MYPET given below. Write a python functions to perform specified operation:

Table : MYPET

Pet_id	Pet_Name	Breed	LifeSpan	Price	Discount
101	Rocky	Labrador Retriever	12	16000	5
202	Duke	German Shepherd	13	22000	10
303	Oliver	Bulldog	10	18000	7
404	Cooper	Yorkshire Terrier	16	20000	12
505	Oscar	Shih Tzu	NULL	25000	8

Create_Table() :- to create the table if doesn't exist, otherwise with proper message.

Add_pet(): - to read input from user and insert the data to the table MYPET.

Display_petdetails(): to read a pet name from user and display its details from table MYPET if it exists in table, otherwise display an appropriate message.

Reduce_Price()-

-Reduces price by the discount whose discount is less than 5.

-Prints the number of rows updated.

Source Code:

```
import mysql.connector

con = mysql.connector.connect(
    host="localhost",
    user="root",
    password="",
    database="PETDB"
)

cur = con.cursor()

# Function to create MYPET table
def Create_Table():
    try:
        q = """CREATE TABLE MYPET(
            PetNo INT PRIMARY KEY,
            PetName VARCHAR(30),
            PetType VARCHAR(20),
            Price FLOAT,
            Discount FLOAT)"""

        cur.execute(q)
        print("Table MYPET created successfully.")

    except:
        print("Table MYPET already exists.")

# Function to add pet details
def Add_pet():
    pno = int(input("Enter Pet Number: "))
    pname = input("Enter Pet Name: ")
    ptype = input("Enter Pet Type: ")
    price = float(input("Enter Price: "))
    discount = float(input("Enter Discount: "))

    q = "INSERT INTO MYPET VALUES({}, '{}', '{}', {}, {})".format(
        pno, pname, ptype, price, discount)

    cur.execute(q)
    con.commit()

    print("Pet details added successfully.")
```

```

# Function to display details of a pet
def Display_petdetails():
    pname = input("Enter Pet Name: ")

    q = "SELECT * FROM MYPET WHERE PetName='{ }'".format(pname)

    cur.execute(q)
    data = cur.fetchall()

    if len(data) == 0:
        print("Pet not found.")

    else:
        print("-" * 65)
        print("{:<10}{:<15}{:<15}{:<15}{:<10}".format(
            "PetNo", "PetName", "PetType", "Price", "Discount"))
        print("-" * 65)

        for row in data:
            print("{:<10}{:<15}{:<15}{:<15}{:<10}".format(
                row[0], row[1], row[2], row[3], row[4]))

        print("-" * 65)

# Function to reduce price
def Reduce_Price():
    q = """UPDATE MYPET
        SET Price = Price - (Price * Discount / 100)
        WHERE Discount < 5"""

    cur.execute(q)
    con.commit()

    print(cur.rowcount, "row(s) updated.")

# Menu-driven program
while True:

    print("\n----- MYPET MENU -----")
    print("1. Create Table")
    print("2. Add Pet")
    print("3. Display Pet Details")
    print("4. Reduce Price")
    print("5. Exit")

    ch = int(input("Enter your choice: "))

    if ch == 1:
        Create_Table()

    elif ch == 2:
        Add_pet()

    elif ch == 3:
        Display_petdetails()

    elif ch == 4:
        Reduce_Price()

    elif ch == 5:
        print("Program terminated.")
        break

    else:
        print("Invalid choice.")

con.close()

```

Output:

Exp No: 7

Date: dd-mm-yyyy

Aim: Write a program in python for Sql connectivity Consider tables emp and dept tables:

- (a) Add a column HRA with default value 2000
- (b) Modify HRA for the employee whose empno is input by the user.
- (c) Display name, job and annual salary of employees whose comm is assigned in descending order of salary.
- (d) Delete the record of employee whose employee number input by the user
- (e) Display empno, name, job and department name who are either a manger or a clerk.
- (f) Display minimum and maximum, sum and average of salary along with number of records in job

Source Code:

```
import mysql.connector

con = mysql.connector.connect(
    host="localhost",
    user="root",
    password="",
    database=""
)

cur = con.cursor()

# (a) Add HRA column with default value 2000
def add_hra():
    try:
        cur.execute("ALTER TABLE emp ADD HRA DECIMAL(10,2) DEFAULT 2000")
        print("HRA column added successfully.")
    except:
        print("HRA column already exists or cannot be added.")

# (b) Modify HRA of an employee
def modify_hra():
    eno = int(input("Enter employee number: "))
    hra = float(input("Enter new HRA: "))

    q = "UPDATE emp SET HRA=%s WHERE empno=%s"
    cur.execute(q, (hra, eno))
    con.commit()

    if cur.rowcount > 0:
        print("HRA updated successfully.")
    else:
        print("Employee not found.")
```

```

# (c) Employees whose commission is assigned
def display_comm():
    q = """SELECT name, job, sal*12 AS annual_salary
           FROM emp
           WHERE comm IS NOT NULL
           ORDER BY sal DESC"""

    cur.execute(q)
    data = cur.fetchall()

    print("{:<20} {:<15} {:>15}".format(
        "NAME", "JOB", "ANNUAL SALARY"))
    print("-" * 52)

    for r in data:
        print("{:<20} {:<15} {:>15.2f}".format(
            r[0], r[1], float(r[2])))

# (d) Delete an employee
def delete_emp():
    eno = int(input("Enter employee number to delete: "))

    q = "DELETE FROM emp WHERE empno=%s"
    cur.execute(q, (eno,))
    con.commit()

    if cur.rowcount > 0:
        print("Employee record deleted successfully.")
    else:
        print("Employee not found.")

# (e) Display Managers and Clerks with department name
def display_job():
    q = """SELECT e.empno, e.name, e.job, d.dname
           FROM emp e, dept d
           WHERE e.deptno = d.deptno
           AND e.job IN ('MANAGER','CLERK')"""

    cur.execute(q)
    data = cur.fetchall()

    print("{:<10} {:<20} {:<15} {:<20}".format(
        "EMPNO", "NAME", "JOB", "DEPARTMENT"))
    print("-" * 68)

    for r in data:
        print("{:<10} {:<20} {:<15} {:<20}".format(
            r[0], r[1], r[2], r[3]))

```

```

# (f) Salary statistics for each job
def salary_details():
    q = """SELECT job, MIN(sal), MAX(sal),
                SUM(sal), AVG(sal), COUNT(*)
            FROM emp
            GROUP BY job"""

    cur.execute(q)
    data = cur.fetchall()

    print("{:<15} {:>10} {:>10} {:>12} {:>12} {:>8}".format(
        "JOB", "MIN", "MAX", "SUM", "AVERAGE", "COUNT"))
    print("-" * 72)

    for r in data:
        print("{:<15} {:>10.2f} {:>10.2f} {:>12.2f} {:>12.2f} {:>8}".format(
            r[0], float(r[1]), float(r[2]),
            float(r[3]), float(r[4]), r[5]))

# Main Menu
while True:
    print("\n===== EMPLOYEE MENU =====")
    print("1. Add HRA column")
    print("2. Modify HRA")
    print("3. Display employees with commission")
    print("4. Delete employee")
    print("5. Display Managers and Clerks")
    print("6. Display salary details job-wise")
    print("7. Exit")

    ch = int(input("Enter your choice: "))

    if ch == 1:
        add_hra()

    elif ch == 2:
        modify_hra()

    elif ch == 3:
        display_comm()

    elif ch == 4:
        delete_emp()

    elif ch == 5:
        display_job()

    elif ch == 6:
        salary_details()

    elif ch == 7:
        break

    else:
        print("Invalid choice")

con.close()

```

Output:

Aim: Write functions for Python –Sql Connectivity:

- (a) List tables in your database
- (b) Create Tables – Garment & Fabric. (Tables given qstn 32)
- (c) Display the structure of the table Fabric
- (d) Insert a Record by accepting values from user
- (e) Delete record of a garment with gcode accepted from the user
- (f) Modify the price of a garment with specific gcode accepted from user.
- (g) Display records of tables by choice -ie., either records of garment or fabric.

Source Code:

```
import mysql.connector

con = mysql.connector.connect(
    host="localhost",
    user="root",
    password="sus123",
    database="ssce24"
)

cur = con.cursor()

# (a) List tables
def list_tables():
    cur.execute("SHOW TABLES")
    data = cur.fetchall()

    print("\nTABLES IN DATABASE")
    for r in data:
        print(r[0])

# (b) Create Garment and Fabric tables
def create_tables():

    q1 = """CREATE TABLE IF NOT EXISTS Fabric(
        fcode VARCHAR(5) PRIMARY KEY,
        type VARCHAR(20))"""

    q2 = """CREATE TABLE IF NOT EXISTS Garment(
        gcode INT PRIMARY KEY,
        description VARCHAR(30),
        price DECIMAL(8,2),
        fcode VARCHAR(5),
        readydate DATE)"""

    cur.execute(q1)
    cur.execute(q2)

    print("Tables created successfully")
```

```

# (c) Display structure of Fabric
def structure():

    cur.execute("DESC Fabric")
    data = cur.fetchall()

    print("\nSTRUCTURE OF FABRIC")

    print("{:<15} {:<20} {:<10} {:<10}".format(
        "FIELD", "TYPE", "NULL", "KEY"))

    print("-" * 55)

    for r in data:
        print("{:<15} {:<20} {:<10} {:<10}".format(
            r[0], r[1], r[2], r[3]))

# (d) Insert record
def insert_record():

    print("\n1. Insert into Garment")
    print("2. Insert into Fabric")

    ch = int(input("Enter choice: "))

    if ch == 1:

        gc = int(input("Enter garment code: "))
        des = input("Enter description: ")
        pr = float(input("Enter price: "))
        fc = input("Enter fabric code: ")
        dt = input("Enter ready date (YYYY-MM-DD): ")

        q = "INSERT INTO Garment VALUES(%s,%s,%s,%s,%s)"

        cur.execute(q, (gc, des, pr, fc, dt))
        con.commit()

        print("Garment record inserted")

    elif ch == 2:

        fc = input("Enter fabric code: ")
        typ = input("Enter fabric type: ")

        q = "INSERT INTO Fabric VALUES(%s,%s)"

        cur.execute(q, (fc, typ))
        con.commit()

        print("Fabric record inserted")

```

```

else:
    print("Invalid choice")

# (e) Delete garment record
def delete_record():

    gc = int(input("Enter garment code to delete: "))

    q = "DELETE FROM Garment WHERE gcode=%s"

    cur.execute(q, (gc,))
    con.commit()

    if cur.rowcount > 0:
        print("Record deleted successfully")
    else:
        print("Garment code not found")

# (f) Modify price
def modify_price():

    gc = int(input("Enter garment code: "))
    pr = float(input("Enter new price: "))

    q = "UPDATE Garment SET price=%s WHERE gcode=%s"

    cur.execute(q, (pr, gc))
    con.commit()

    if cur.rowcount > 0:
        print("Price modified successfully")
    else:
        print("Garment code not found")

# (g) Display records
def display_records():

    print("\n1. Display Garment")
    print("2. Display Fabric")

    ch = int(input("Enter choice: "))

    if ch == 1:

        cur.execute("SELECT * FROM Garment")
        data = cur.fetchall()

        print("\nGARMENT TABLE")

        print("{:<8} {:<20} {:<10} {:<8} {:<12}".format(
            "GCODE", "DESCRIPTION", "PRICE",
            "FCODE", "READYDATE"))

        print("-" * 65)

        for r in data:
            print("{:<8} {:<20} {:<10} {:<8} {:<12}".format(
                r[0], r[1], r[2], r[3], str(r[4])))

```

```

elif ch == 2:

    cur.execute("SELECT * FROM Fabric")
    data = cur.fetchall()

    print("\nFABRIC TABLE")

    print("{:<10} {:<20}".format(
        "FCODE", "TYPE"))

    print("-" * 30)

    for r in data:
        print("{:<10} {:<20}".format(
            r[0], r[1]))

else:
    print("Invalid choice")
while True:

    print("\n===== GARMENT DATABASE =====")

    print("1. List Tables")
    print("2. Create Tables")
    print("3. Display Structure of Fabric")
    print("4. Insert Record")
    print("5. Delete Garment Record")
    print("6. Modify Garment Price")
    print("7. Display Records")
    print("8. Exit")

    ch = int(input("Enter your choice: "))

    if ch == 1:
        list_tables()

    elif ch == 2:
        create_tables()

    elif ch == 3:
        structure()

    elif ch == 4:
        insert_record()

    elif ch == 5:
        delete_record()

    elif ch == 6:
        modify_price()

    elif ch == 7:
        display_records()

    elif ch == 8:
        print("Program terminated")
        break

    else:
        print("Invalid choice")

```

Output:

Aim: A list of names is stored in a text file, **Names.txt**, (1 name per line) but it contains lot of duplicates. Create a Python function `extract_unique_name()` that extracts all unique names from the text file and stores them in a list and then print the names in alphabetical order.

Sample input file content:

```
Amithab
Sharuk
Deepika
Prabhas
amithab
DEEPIKA
```

Expected output:

```
['amithab', 'deepika', 'prabhas', 'sharuk']
```

Source Code:

```
def create_file():
    f = open("Names.txt", "w")

    f.write("Amithab\n")
    f.write("Sharuk\n")
    f.write("Deepika\n")
    f.write("Prabhas\n")
    f.write("amithab\n")
    f.write("DEEPIKA\n")

    f.close()

def extract_unique_name():
    f = open("Names.txt", "r")

    names = []

    for line in f:
        name = line.strip().lower()

        if name not in names:
            names.append(name)

    f.close()

    names.sort()
    print(names)

create_file()
extract_unique_name()
```

Output:

Aim: Write a program with user defined functions

CreateWord()- to create '**word.txt**' file – by accepting n of lines of text from user and write it into file using **writelines()**.

DisplayFile()- Display number of characters and number of words in it.

Source Code:

```
def CreateWord():
    n = int(input("Enter number of lines: "))
    lines = []

    for i in range(n):
        s = input("Enter line: ")
        lines.append(s + "\n")

    f = open("word.txt", "w")
    f.writelines(lines)
    f.close()

def DisplayFile():
    f = open("word.txt", "r")
    data = f.read()
    f.close()

    char_count = len(data)
    word_count = len(data.split())

    print("Number of characters =", char_count)
    print("Number of words =", word_count)
```

```
CreateWord()
DisplayFile()
```

Output:

Aim: Write a program with user defined functions

CreateStory()- n lines accepted from user in a text file **story** with few lines in it.

DisplayWord()- Display each word separated by a '#'

```
def CreateStory():
    n = int(input("Enter number of lines: "))

    f = open("story.txt", "w")

    for i in range(n):
        line = input("Enter line: ")
        f.write(line + "\n")

    f.close()

def DisplayWord():
    f = open("story.txt", "r")

    data = f.read()
    words = data.split()

    f.close()

    print("#".join(words))
```

```
CreateStory()
DisplayWord()
```

Aim: Write a program with user defined functions

CreatePledge()- to create **pledge.txt**, containing few lines of pledge.

DisplayPledge()- Display the contents line by line (after stripping leading and trailing whitespaces) along with number of characters in each line

```
def CreatePledge():
    n = int(input("Enter number of lines: "))

    f = open("pledge.txt", "w")

    for i in range(n):
        line = input("Enter line: ")
        f.write(line + "\n")

    f.close()

def DisplayPledge():
    f = open("pledge.txt", "r")

    for line in f:
        line = line.strip()
        print(line, "-", len(line), "characters")

    f.close()

CreatePledge()
DisplayPledge()
```

Aim: Write a program with user defined functions

CreatePoem()- to create the file poem.txt containing the following :

The woods are lovely

dark and deep

But I have promises to keep

And miles to go before I sleep

And miles to go before I sleep (Note: all lines must be user input)

DisplayPoem()- displays the n most commonly occurring words.

```
def CreatePoem():
    f = open("poem.txt", "w")

    print("Enter the 5 lines of the poem:")

    for i in range(5):
        line = input()
        f.write(line + "\n")

    f.close()

def DisplayPoem():
    f = open("poem.txt", "r")

    data = f.read()
    f.close()

    data = data.lower()
    words = data.split()

    # Create frequency dictionary
    freq = {}

    for word in words:
        if word in freq:
            freq[word] = freq[word] + 1
        else:
            freq[word] = 1

    print("\nFrequency Dictionary:")
    print(freq)
```

```

# Convert dictionary into list of tuples
items = list(freq.items())

# Sort according to frequency in descending order
for i in range(len(items) - 1):
    for j in range(i + 1, len(items)):

        if items[i][1] < items[j][1]:
            temp = items[i]
            items[i] = items[j]
            items[j] = temp

# Create sorted dictionary
sorted_freq = {}

for item in items:
    word = item[0]
    count = item[1]

    sorted_freq[word] = count

print("\nDictionary sorted by frequency:")
print(sorted_freq)

n = int(input("\nEnter number of most common words: "))

print("\nMost commonly occurring words:")

count = 0

for word in sorted_freq:

    if count < n:
        print(word, "-", sorted_freq[word])
        count = count + 1
    else:
        break

```

```

CreatePoem()
DisplayPoem()

```

Exp No: 14

Date: dd-mm-yyyy

Aim: Write user-defined functions:

createText() – create the text file wisdom.txt with the following content.

My son, listen to your father's advice.

Don't turn away from your mother's teaching.

What they teach you will be like a beautiful crown on your head.

It will be like a chain to decorate your neck.

(Note: all lines must be user input)

NoVowels() reads the content of the file wisdom.txt and display those words that doesn't contain any vowels in it.

vowelWord()- reads the content of the file wisdom.txt displays those lines which have the second letter of the first word a vowel.

```

def createText():
    f = open("wisdom.txt", "w")
    n=int(input("Enter number of lines:"))
    for i in range(n):
        line = input("Enter line: ")
        f.write(line + "\n")

    f.close()

def NoVowels():
    f = open("wisdom.txt", "r")
    data = f.read()
    f.close()

    words = data.split()
    print("Words with no vowels:")
    for word in words:
        flag = True

        for ch in word.lower():
            if ch in "aeiou":
                flag = False
                break

        if flag:
            print(word)

def vowelWord():
    f = open("wisdom.txt", "r")
    print("Words with vowels:")
    for line in f:
        words = line.split()

        if len(words) > 0:
            first = words[0]

            if len(first) >= 2 and first[1].lower() in "aeiou":
                print(line.strip())

    f.close()

#createText()
NoVowels()
vowelWord()

```

Aim: Write a program with user defined functions

- crtBookFile()**- for entering details of books from user and insert it in a binary file named '**book.det**' through list.
Sample: [Bookid, Bookname, Authorname, Price]
- BookPrice()** -to display book details having price greater than a given price.
- SearchBook()**- Search and display the details of a specific book entered by user

```
import pickle
# (a) Create binary file
def crtBookFile():
    f = open("book.det", "wb")

    n = int(input("Enter number of books: "))

    for i in range(n):
        bid = int(input("Enter Book ID: "))
        bname = input("Enter Book Name: ")
        author = input("Enter Author Name: ")
        price = float(input("Enter Price: "))

        book = [bid, bname, author, price]

        pickle.dump(book, f)

    f.close()
# (b) Display books above a given price
def BookPrice():
    f = open("book.det", "rb")

    p = float(input("Enter price: "))

    try:
        while True:
            book = pickle.load(f)

            if book[3] > p:
                print(book)

    except EOFError:
        f.close()
# (c) Search for a specific book
def SearchBook():
    f = open("book.det", "rb")

    name = input("Enter Book Name to search: ")
    found = False

    try:
        while True:
            book = pickle.load(f)

            if book[1].lower() == name.lower():
                print(book)
                found = True

    except EOFError:
        f.close()

    if found == False:
        print("Book not found")

crtBookFile()
BookPrice()
SearchBook()
```

Aim: Write a user defined function:

- MakeFile ()** to input multiple items from the user and add to binary file **Items.dat** which has structure as [Code, Description, Price].
- SearchRec(Code)** in Python which will accept the code as parameter and search and display the details of the corresponding code on screen from Items.dat. Also display number items having such code out of total number of items.

```
import pickle

# Function to create binary file
def MakeFile():
    f = open("Items.dat", "ab")

    while True:
        code = int(input("Enter item code: "))
        desc = input("Enter description: ")
        price = float(input("Enter price: "))

        rec = [code, desc, price]
        pickle.dump(rec, f)

        ch = input("Add another item? (Y/N): ")
        if ch.upper() == "N":
            break

    f.close()

# Function to search for an item code
def SearchRec(Code):
    f = open("Items.dat", "rb")

    total = 0
    count = 0

    try:
        while True:
            rec = pickle.load(f)
            total += 1

            if rec[0] == Code:
                print(rec)
                count += 1

    except EOFError:
        f.close()

    if count == 0:
        print("Item not found")

    print("Number of items having code", Code, "=", count)
    print("Total number of items =", total)

# Function calls
MakeFile()

c = int(input("Enter code to search: "))
SearchRec(c)
```

Aim: Write a program with user defined functions

- CreatEmp()**- for entering details of employees from user and insert it in a binary file named 'emp.dat' through dictionary. Sample record has the form {empno:[empname,job, sal],
Sample: {101:[Mark,Manager,2000]}
- DisplayEmps()**-Display the records of all employees in tabular form.
- SearchEmp()**-display those employees' details whose salary is between 25000 and 30000
- SearchEmpno(eno)**- accept the argument and display the record of the employee whose empno is input by the user.

```
import pickle

# a) Create employee records
def CreatEmp():
    f = open("emp.dat", "ab")

    while True:
        eno = int(input("Enter employee number: "))
        name = input("Enter employee name: ")
        job = input("Enter job: ")
        sal = float(input("Enter salary: "))

        rec = {eno: [name, job, sal]}
        pickle.dump(rec, f)

        ch = input("Add another employee? (Y/N): ")
        if ch.upper() == "N":
            break

    f.close()

# b) Display all employee records
def DisplayEmps():
    f = open("emp.dat", "rb")

    print("\nEmpNo\tName\t\tJob\t\tSalary")
    print("-" * 55)

    try:
        while True:
            rec = pickle.load(f)

            for eno in rec:
                data = rec[eno]
                print(eno, "\t", data[0], "\t\t",
                      data[1], "\t\t", data[2])

    except EOFError:
        f.close()
```

```

# c) Display employees having salary between 25000 and 30000
def SearchEmp():
    f = open("emp.dat", "rb")
    found = False

    print("\nEmployees with salary between 25000 and 30000")
    print("EmpNo\tName\t\tJob\t\tSalary")
    print("-" * 55)

    try:
        while True:
            rec = pickle.load(f)

            for eno in rec:
                data = rec[eno]

                if 25000 <= data[2] <= 30000:
                    print(eno, "\t", data[0], "\t\t",
                        data[1], "\t\t", data[2])
                    found = True

    except EOFError:
        f.close()

    if found == False:
        print("No employee found")

# Function calls
CreatEmp()
DisplayEmps()
SearchEmp()

eno = int(input("\nEnter employee number to search: "))
SearchEmpno(eno)

```

Aim: A Binary file, CINEMA.DAT has the following structure:

```
{'MNO':'m101','MNAME':'Sound of Freedom','MTYPE':'Thriller'}}
```

Where, MNO – Movie Number; MNAME – Movie Name; MTYPE is Movie Type

Write a user defined function:

createCinema(): that Adds records the user input data into the binary file.

findType(mtype): that accepts mtype as parameter and displays all the records from the binary file CINEMA.DAT, that have the value of Movie Type as mtype.

```
import pickle

# Function to add movie records
def createCinema():
    f = open("CINEMA.DAT", "ab")

    while True:
        mno = input("Enter movie number: ")
        mname = input("Enter movie name: ")
        mtype = input("Enter movie type: ")

        rec = {'MNO': mno, 'MNAME': mname, 'MTYPE': mtype}
        pickle.dump(rec, f)

        ch = input("Add another movie? (Y/N): ")
        if ch.upper() == "N":
            break

    f.close()

# Function to display movies of a given type
def findType(mtype):
    f = open("CINEMA.DAT", "rb")
    found = False

    try:
        while True:
            rec = pickle.load(f)

            if rec['MTYPE'].lower() == mtype.lower():
                print(rec)
                found = True

    except EOFError:
        f.close()

    if found == False:
        print("No movie found with type", mtype)

# Function calls
createCinema()

typ = input("Enter movie type to search: ")
findType(typ)
```

Aim: A file, PASSENGERS.DAT, stores the records of passengers using the following structure:

[PNR, PName, BRDSTN, DESTN, FARE]

where:

- PNR** - Passenger Number (string type)
- PName** - Passenger Name (string type)
- BRDSTN** - Boarding Station Name (string type)
- DESTN** - Destination Station Name (string type)
- FARE** - Fare amount for the journey (float type)

Write user defined functions in Python for the following tasks:

- i. **Create()** - to input data for passengers and write it in the binary file PASSENGERS.DAT.
- ii. **SearchDestn(D)** - to read contents from the file PASSENGERS.DAT and display the details of those Passengers whose DESTN matches with the value of D.
- (iii) **UpdateFare ()** - to increase the fare of all passengers by 5% and rewrite the updated records into the file PASSENGERS.DAT.
- (iv) **DeleteData()** - to delete the details of a passenger whose PNR is input by user.

```
import pickle

# i) Create passenger records
def Create():
    f = open("PASSENGERS.DAT", "ab")

    while True:
        pnr = input("Enter PNR: ")
        name = input("Enter passenger name: ")
        brd = input("Enter boarding station: ")
        dest = input("Enter destination: ")
        fare = float(input("Enter fare: "))

        rec = [pnr, name, brd, dest, fare]
        pickle.dump(rec, f)

        ch = input("Add another passenger? (Y/N): ")
        if ch.upper() == "N":
            break

    f.close()

# ii) Search passengers by destination
def SearchDestn(D):
    f = open("PASSENGERS.DAT", "rb")
    found = False

    try:
        while True:
            rec = pickle.load(f)

            if rec[3].lower() == D.lower():
                print(rec)
                found = True

    except EOFError:
        f.close()

    if found == False:
        print("No passenger found")
```

```

# iii) Increase fare of all passengers by 5%
def UpdateFare():
    f = open("PASSENGERS.DAT", "rb")
    data = []

    try:
        while True:
            rec = pickle.load(f)
            rec[4] = rec[4] + rec[4] * 5 / 100
            data.append(rec)

    except EOFError:
        f.close()

    f = open("PASSENGERS.DAT", "wb")

    for rec in data:
        pickle.dump(rec, f)

    f.close()

    print("Fare updated successfully")

# iv) Delete passenger using PNR
def DeleteData():
    p = input("Enter PNR to delete: ")

    f = open("PASSENGERS.DAT", "rb")
    data = []
    found = False

    try:
        while True:
            rec = pickle.load(f)

            if rec[0] == p:
                found = True
            else:
                data.append(rec)

    except EOFError:
        f.close()

    f = open("PASSENGERS.DAT", "wb")

    for rec in data:
        pickle.dump(rec, f)

    f.close()

    if found:
        print("Passenger record deleted")
    else:
        print("PNR not found")

```

```
# Function calls
```

```
Create()
```

```
d = input("Enter destination to search: ")  
SearchDestn(d)
```

```
UpdateFare()
```

```
DeleteData()
```

Exp No: 20

Date: dd-mm-yyyy

Aim: Order data (order_id, customer_name, order_date, total_amount) is stored in a binary file named Orders.dat. Each order is represented as a tuple. Help Simran complete the following tasks:

Write user-defined functions:

add_order() - to input order details from the user (order_id, customer_name, order_date, total_amount) and store them in "Orders.dat". The program should allow adding multiple orders until the user chooses to stop.

update_order_amount()- to modify the total_amount-for orders placed. The function should increase the total_amount of each qualifying order by 10%.

```
import pickle
```

```
# Function to add orders
```

```
def add_order():
```

```
    f = open("Orders.dat", "ab")
```

```
    while True:
```

```
        oid = int(input("Enter order id: "))
```

```
        name = input("Enter customer name: ")
```

```
        date = input("Enter order date: ")
```

```
        amount = float(input("Enter total amount: "))
```

```
        rec = (oid, name, date, amount)
```

```
        pickle.dump(rec, f)
```

```
        ch = input("Add another order? (Y/N): ")
```

```
        if ch.upper() == "N":
```

```
            break
```

```
    f.close()
```

```

# Function to increase order amount by 10%
def update_order_amount():
    f = open("Orders.dat", "rb")
    data = []

    try:
        while True:
            rec = pickle.load(f)

            oid, name, date, amount = rec
            amount = amount + amount * 10 / 100

            rec = (oid, name, date, amount)
            data.append(rec)

    except EOFError:
        f.close()

    f = open("Orders.dat", "wb")

    for rec in data:
        pickle.dump(rec, f)

    f.close()

    print("Order amounts updated successfully")

# Function calls
add_order()
update_order_amount()

```

Exp No: 21

Date: dd-mm-yyyy

Aim: Consider book inventory using a CSV file named Inventory.csv.

The file structure is:

[BookID, Title, Author, Available]

where

BookID is an integer

Title and Author are strings

Available is an integer representing the number of copies available.

The librarian needs to write the following functions: -

- a) `add_book()`: This function accepts new book details from the user and adds them to 'Inventory.csv'. The file should be created with column header if it doesn't exist.
- b) `check_availability(book_id)`: This function takes a `book_id` as input and returns the number of copies available for that book. If the book is not found, it should return -1.

```

import csv

# a) Add a new book
def add_book():
    f = open("Inventory.csv", "a+", newline="")
    f.seek(0)

    # Write header if file is empty
    if f.read() == "":
        f.seek(0)
        w = csv.writer(f)
        w.writerow(["BookID", "Title", "Author", "Available"])

    f.seek(0, 2)

    bid = int(input("Enter Book ID: "))
    title = input("Enter Title: ")
    author = input("Enter Author: ")
    available = int(input("Enter number of copies available: "))

    w = csv.writer(f)
    w.writerow([bid, title, author, available])

    f.close()

# b) Check number of available copies
def check_availability(book_id):
    f = open("Inventory.csv", "r", newline="")
    r = csv.reader(f)

    next(r)    # Skip header

    for rec in r:
        if int(rec[0]) == book_id:
            f.close()
            return int(rec[3])

    f.close()
    return -1

# Function calls
add_book()

bid = int(input("Enter Book ID to check: "))
n = check_availability(bid)

if n == -1:
    print("Book not found")
else:
    print("Available copies:", n)

```

Aim: Assuming a CSV file students.csv has columns like student_id, name, class, div, subject, score and grade_level. Write the definition of functions to perform the following.

- Add_data(filename) :- accepts a file name as argument and reads input from user and store data into the file as per format specified.
- Display_data(filename) :- accepts a filename as argument and reads data from file. Function also reads subject name, class and div as input from user. Displays the average score by the student for the subject in the class specified.

```
import csv

# a) Add student data to CSV file
def Add_data(filename):
    f = open(filename, "a+", newline="")
    f.seek(0)

    # Add header if file is empty
    if f.read() == "":
        f.seek(0)
        w = csv.writer(f)
        w.writerow(["student_id", "name", "class", "div",
                    "subject", "score", "grade_level"])

    f.seek(0, 2)
    w = csv.writer(f)

    while True:
        sid = int(input("Enter Student ID: "))
        name = input("Enter Name: ")
        cls = input("Enter Class: ")
        div = input("Enter Division: ")
        subject = input("Enter Subject: ")
        score = float(input("Enter Score: "))
        grade = input("Enter Grade Level: ")

        w.writerow([sid, name, cls, div, subject, score, grade])

        ch = input("Add another record? (Y/N): ")
        if ch.upper() == "N":
            break

    f.close()
```

b) Display average score for a subject, class and division

```
def Display_data(filename):
    subject = input("Enter Subject: ")
    cls = input("Enter Class: ")
    div = input("Enter Division: ")

    f = open(filename, "r", newline="")
    r = csv.reader(f)

    next(r)          # Skip header

    total = 0
    count = 0

    for rec in r:
        if (rec[4].lower() == subject.lower() and
            rec[2] == cls and
            rec[3].lower() == div.lower()):

            total = total + float(rec[5])
            count = count + 1

    f.close()

    if count > 0:
        avg = total / count
        print("Average Score =", avg)
    else:
        print("No matching records found")
```

Function calls

Add_data("students.csv")

Display_data("students.csv")

Aim: Anil created a csv file named gadgets.csv to store the details of the gadgets

The structure of gadgets.csv is:

[Deviceno, Name, Price, Brand] - Where, Deviceno is device ID (integer), Name is device name (string), Price is device price (integer), Brand is device brand (string)

He is now instructed to write the following functions to perform the required operations on csv files.

Newgadget() -to add the details of new gadgets in csv file gadget.csv which stores records in the format. Deviceno, name, price, brand. Get the input from the user.

Countgadget() - to read the csv file „gadget.csv“ and display the details of the devices whose price in the range 1000 to 2500. Also, display the number of such devices.

```
import csv

# Function to add new gadget details
def Newgadget():
    f = open("gadget.csv", "a", newline="")
    w = csv.writer(f)

    while True:
        dno = int(input("Enter Device Number: "))
        name = input("Enter Device Name: ")
        price = int(input("Enter Price: "))
        brand = input("Enter Brand: ")

        w.writerow([dno, name, price, brand])

        ch = input("Add another gadget? (Y/N): ")
        if ch.upper() == "N":
            break

    f.close()

# Function to display gadgets priced between 1000 and 2500
def Countgadget():
    f = open("gadget.csv", "r", newline="")
    r = csv.reader(f)

    count = 0

    print("DeviceNo\tName\tPrice\tBrand")
    print("-" * 40)

    for rec in r:
        if 1000 <= int(rec[2]) <= 2500:
            print(rec[0], "\t", rec[1], "\t", rec[2], "\t", rec[3])
            count = count + 1

    f.close()

    print("Number of gadgets =", count)

# Function calls
Newgadget()
Countgadget()
```

Aim: Samantha, a software developer at a company, is tasked with managing a database for employee performance reviews. She has created a CSV file named "Performance.csv" to store the data. The structure of "Performance.csv" is as follows:

[Emp_Id, Emp_Name, Project_Name, Review_Result]

Where:

Emp_Id - employee ID (integer)

Emp_Name - employee name (string)

Project_Name - name of the project for the performance review (string)

Review_Result - result of the review, which can be 'Excellent', 'Satisfactory', or 'Needs Improvement'

Samantha needs to implement the following user-defined functions to efficiently handle the data:

`accept_record()` - This function should accept a record from the user, including employee ID, employee name, project name, and review result, and add it to the "Performance.csv" file. The column headings should also be added on top of the CSV file.

`excellent_count()` - Samantha wants a function to count the number of employees who received an 'Excellent' review for any project. Help Samantha by providing Python code for these user-defined functions

```
import csv

# Function to accept and add employee performance record
def accept_record():
    f = open("Performance.csv", "a+", newline="")
    f.seek(0)

    # Add column headings if file is empty
    if f.read() == "":
        f.seek(0)
        w = csv.writer(f)
        w.writerow(["Emp_Id", "Emp_Name", "Project_Name", "Review_Result"])

    f.seek(0, 2)
    w = csv.writer(f)

    eid = int(input("Enter Employee ID: "))
    name = input("Enter Employee Name: ")
    project = input("Enter Project Name: ")
    result = input("Enter Review Result: ")

    w.writerow([eid, name, project, result])

    f.close()
```

```

# Function to count Excellent reviews
def excellent_count():
    f = open("Performance.csv", "r", newline="")
    r = csv.reader(f)

    next(r)          # Skip header
    count = 0

    for rec in r:
        if rec[3].lower() == "excellent":
            count = count + 1

    f.close()

    print("Number of employees with Excellent review =", count)

# Function calls
accept_record()
excellent_count()

```

Aim: Write a Python menu driven program to implement a stack (LIFO) using a list data structure.

A linear stack called Books contains the following information:

[BookNumber, name of the book and cost of the book]

- PUSH(Books,N) method in python to add N book details each containing the above mentioned information's.
- DispStack()- display the stack content in same line
- POP(Book)- pop and display all the elements of the stack and **Under Flow** when no more elements to be popped.

```
Books = []
```

```
# Function to push N books into stack
```

```
def PUSH(Books, N):  
    for i in range(N):  
        bno = int(input("Enter Book Number: "))  
        name = input("Enter Book Name: ")  
        cost = float(input("Enter Cost: "))  
  
        book = [bno, name, cost]  
        Books.append(book)
```

```
# Function to display stack contents in same line
```

```
def DispStack():  
    if len(Books) == 0:  
        print("Stack is empty")  
    else:  
        for book in Books:  
            print(book, end=" ")  
        print()
```

```
# Function to pop and display all elements
```

```
def POP(Books):  
    if len(Books) == 0:  
        print("Under Flow")  
    else:  
        while len(Books) > 0:  
            print("Popped:", Books.pop())  
  
        print("Under Flow")
```

```

# Menu driven program
while True:
    print("\n1. PUSH")
    print("2. DISPLAY STACK")
    print("3. POP")
    print("4. EXIT")

    ch = int(input("Enter your choice: "))

    if ch == 1:
        n = int(input("Enter number of books: "))
        PUSH(Books, n)

    elif ch == 2:
        DispStack()

    elif ch == 3:
        POP(Books)

    elif ch == 4:
        print("Program ended")
        break

    else:
        print("Invalid choice")

```

Exp No: 26

Date: dd-mm-yyyy

Aim: A list of dictionaries **contact** contains the following information:

{Eid:[Ename, Phno,Salary]} of Employee

Write a Python menu driven program to implement a stack (LIFO) using a list data structure, stack

status :

PUSH () - To Push an object containing Phone number of Employee and Name of Employee into the stack.

POP() - To Pop an object from the stack and to release the memory

Display() – display details of all Employees

contact is a list of dictionaries

```

contact = [
    {101: ["Arun", 9876543210, 35000]},
    {102: ["Meera", 9123456780, 42000]},
    {103: ["Rahul", 9988776655, 38000]}
]

status = []

def PUSH():
    for d in contact:
        for eid in d:
            ename = d[eid][0]
            phno = d[eid][1]
            status.append([ename, phno])
    print("Employee details pushed into stack")

def POP():
    if status == []:
        print("Stack Underflow")
    else:
        x = status.pop()
        print("Deleted:", x)

```

```

def Display():
    if status == []:
        print("Stack is Empty")
    else:
        print("Name\tPhone Number")
        for i in range(len(status)-1, -1, -1):
            print(status[i][0], "\t", status[i][1])

while True:
    print("\n1. PUSH")
    print("2. POP")
    print("3. DISPLAY")
    print("4. EXIT")

    ch = int(input("Enter your choice: "))

    if ch == 1:
        PUSH()
    elif ch == 2:
        POP()
    elif ch == 3:
        Display()
    elif ch == 4:
        break
    else:
        print("Invalid Choice")

```

Exp No: 27

Date: dd-mm-yyyy

Aim: Write a function in Python where Book is a stack implemented by a list of books:

(i) **PushBook()** - to add a new book entry as a record – ie. (book_no, book_title) of Books , considering it to act as push operations of the Stack data structure.

(ii) **Disp_Stack()** – display details of all books

Bookno: Booktitle one after other in different lines

(iii) **PopBook()** – display a record removed from the stack in the following format:

Bookno: Booktitle

Book = []

(i) Push a new book into the stack

```

def PushBook():
    bno = int(input("Enter Book Number: "))
    title = input("Enter Book Title: ")

    rec = (bno, title)
    Book.append(rec)

```

(ii) Display all books in the stack

```

def Disp_Stack():
    print("Display all books in the stack")
    if len(Book) == 0:
        print("Stack is empty")
    else:
        for rec in Book:
            print(rec[0], ":", rec[1])

```

```

# (iii) Pop and display one book from the stack
def PopBook():
    print("Pop and display one book from the stack")
    if len(Book) == 0:
        print("Under Flow")
    else:
        rec = Book.pop()
        print(rec[0], ":", rec[1])

# Function calls
n=int(input("Number of books:"))
for i in range(n):
    PushBook()

Disp_Stack()

PopBook()

```

Exp No: 28

Date: dd-mm-yyyy

Aim: Julie has created a dictionary containing names and marks as key value pairs of 6 students. Write a program, with separate user defined functions to perform the following operations:

- PUSH(R)-Push the keys (name of the student) of the dictionary into a stack, where the corresponding value (marks) is greater than 75.
- POP()- Pop and display the content of the stack with 'Under flow' message when there are no more elements to be popped.

```

S = []

# Push names having marks greater than 75
def PUSH(R):
    for name in R:
        if R[name] > 75:
            S.append(name)

# Pop and display all elements
def POP():
    while len(S) > 0:
        print(S.pop(), end=" ")

    print("Under flow")

# Main program
R = {"OM":76, "JAI":45, "BOB":89, "ALI":65, "ANU":90, "TOM":82}

PUSH(R)
POP()

```

Aim: Andrew has a list containing 10 integers. You need to help him create a program with separate user defined functions to perform the following operations based on this list.

- PUSHNums(N)- Traverse the content of the list and push the even numbers into a stack.
- PopNums()- pop and display the content of the stack.

S = []

Push even numbers into stack

def PUSHNums(N):

for num in N:

if num % 2 == 0:

S.append(num)

Pop and display stack contents

def PopNums():

while len(S) > 0:

print(S.pop(), end=" ")

print("Empty Stack")

Main program

N = [12, 13, 34, 56, 21, 79, 98, 22, 35, 38]

PUSHNums(N)

PopNums()

Exp No: 30

Date: dd-mm-yyyy

Aim: Write a function in Python, where Arr is a stack implemented by a list of numbers:

- PUSH(L), where L is a list of numbers. From this list push all numbers divisible by 5 into a stack implemented by using a list.
- Display the stack if it has at least one element, otherwise display appropriate error
- POP(), pop and returns top element when ever it is called, returns 'UnderFlow' if the stack is empty.

Arr = []

(i) Push numbers divisible by 5 into stack

def PUSH(L):

for n in L:

if n % 5 == 0:

Arr.append(n)

(ii) Display stack

def Display():

if len(Arr) > 0:

print(Arr)

else:

print("Stack is empty")

```

# (iii) Pop and return top element
def POP():
    if len(Arr) == 0:
        return "UnderFlow"
    else:
        return Arr.pop()

# Main program
L = [12, 25, 30, 43, 50, 66, 75]

PUSH(L)
Display()

print(POP())

```

Exp No: 31

Date: dd-mm-yyyy

Aim: Consider tables Garment and Fabric.

Table: Fabric

FCODE	TYPE
F01	Terelene
F02	Cotton
F03	Silk
F04	Polyster
F05	Jute
F06	Net

6 rows in set (0.05 sec)

Table: Garment

GCODE	Description	PRICE	FCODE	READYDATE
10001	Formal Shirt	1250	F01	2008-01-12
10007	Formal Pant	1350	F01	2008-03-09
10009	Informal Pant	1500	F02	2008-10-20
10012	Informal Shirt	1550	F02	1008-06-06
10019	Evening Gown	850	F03	2008-06-06
10020	Frock	750	F04	2007-09-09
10023	Pencil Skirt	1150	F03	2008-12-19
10024	Baby Top	750	F03	2007-04-07
10025	Saree	3500	F03	2009-09-15
10026	Parallels	2300	F01	2008-02-17
10089	SLACKS	900	F03	2008-10-31
10090	Tulip Skirt	850	F02	2007-03-31

12 rows in set (0.17 sec)

- Display min, max, sum, and average price for each fabric codes in table Garment.
- Find the total number of garments in each Type.
- Increase the price of 'SLACKS' by 150.
- Display fabric code, description, price and type of all garments
- Display Maximum Fabric code, minimum Ready date.
- Display different Fabric codes.
- Display number of different Fabric codes.
- Display the name of garments whose length contains less than 7 characters.
- Display the name of fabrics ready in 2008.
- Display Equi-join
- Display natural join.

```
CREATE TABLE Fabric
```

```
( FCODE VARCHAR(3) PRIMARY KEY,
  TYPE VARCHAR(20));
```

```
CREATE TABLE Garment(
```

```
  GCODE INT PRIMARY KEY,
  Description VARCHAR(30),
  PRICE INT,
  FCODE VARCHAR(3),
  READYDATE DATE,
  FOREIGN KEY (FCODE) REFERENCES Fabric(FCODE));
```

```
INSERT INTO Fabric VALUES
```

```
('F01', 'Terelene'),('F02', 'Cotton'),('F03', 'Silk'),('F04', 'Polyster'),('F05', 'Jute'),('F06', 'Net');
```

INSERT INTO Garment VALUES

(10001, 'Formal Shirt', 1250, 'F01', '2008-01-12'),
(10007, 'Formal Pant', 1350, 'F01', '2008-03-09'),
(10009, 'Informal Pant', 1500, 'F02', '2008-10-20'),
(10012, 'Informal Shirt', 1550, 'F02', '2008-06-06'),
(10019, 'Evening Gown', 850, 'F03', '2008-06-06'),
(10020, 'Frock', 750, 'F04', '2007-09-09'),
(10023, 'Pencil Skirt', 1150, 'F03', '2008-12-19'),
(10024, 'Baby Top', 750, 'F03', '2007-04-07'),
(10025, 'Saree', 3500, 'F03', '2009-09-15'),
(10026, 'Parallels', 2300, 'F01', '2008-02-17'),
(10089, 'SLACKS', 900, 'F03', '2008-10-31'),
(10090, 'Tulip Skirt', 850, 'F02', '2007-03-31');

a) SELECT FCODE, MIN(PRICE), MAX(PRICE), SUM(PRICE), AVG(PRICE)
FROM Garment
GROUP BY FCODE;

b) SELECT TYPE, COUNT(GCODE)
FROM Fabric F, Garment G
WHERE F.FCODE = G.FCODE
GROUP BY TYPE;

c) UPDATE Garment
SET PRICE = PRICE + 150
WHERE Description = 'SLACKS';

d) SELECT G.FCODE, Description, PRICE, TYPE
FROM Garment G, Fabric F
WHERE G.FCODE = F.FCODE;

e) SELECT MAX(FCODE), MIN(READYDATE)
FROM Garment;

f) SELECT DISTINCT FCODE

FROM Garment;

g) SELECT COUNT(DISTINCT FCODE)

FROM Garment;

h) SELECT Description

FROM Garment

WHERE LENGTH(Description) < 7;

i) SELECT Description

FROM Garment

WHERE YEAR(READYDATE) = 2008;

j) SELECT *

FROM Garment G, Fabric F

WHERE G.FCODE = F.FCODE;

k) SELECT *

FROM Garment NATURAL JOIN Fabric;

Exp No: 32

Date: dd-mm-yyyy

Aim: Consider the table **Student**

id	name	ENG	PHY	CHE	MATHS	CS
101	Megan	NULL	NULL	NULL	NULL	NULL
s101	Abhishek	99	98	94	78	100
s102	Bhuwaneshvar	96	99	94	78	97
s103	Mohan	95	100	100	100	100
s104	Catherine	95	76	NULL	74	63
s105	Dominic	57	34	33	54	70
s106	Ellen	21	18	22	15	34

7 rows in set (0.00 sec)

a) Modify the id 101 as **s107**

b) Add column Tot

- c) Modify the values in column Tot as sum of all subjects(ie., Eng+Phy+Che+Maths+CS) using a query.
- d) Add columns Average and Grade.
- e) Modify the values in column Average with Tot/5, assign **A+** in column **Grade** for those students whose **average** mark greater than 91. (using Queries)
- f) Delete the details of a student in the table whose Tot mark is not assigned.
- g) Display the details of the students whose average mark more than 80.
- h) Display the min, max, sum, and average of CS marks.
- i) Display student ID, Name &marks arranged in descending order of the Average mark.
- j) Add the record - 's104','Catherine', 95,76, NULL, 74, 63.
- k) Change Che =80 and Grade ='A' for the student with id s104.

```
CREATE TABLE Student(
  id VARCHAR(4) PRIMARY KEY,
  name VARCHAR(20),
  ENG INT,
  PHY INT,
  CHE INT,
  MATHS INT,
  CS INT);
```

```
INSERT INTO Student VALUES
('101','Megan',NULL,NULL,NULL,NULL,NULL),
('s101','Abhishek',99,98,94,78,100),
('s102','Bhuvaneshvar',96,99,94,78,97),
('s103','Mohan',95,100,100,100,100),
('s104','Catherine',95,76,NULL,74,63),
('s105','Dominic',57,34,33,54,70),
('s106','Ellen',21,18,22,15,34);
```

-- a) Modify id 101 as s107

```
UPDATE Student
SET id='s107'
WHERE id='101';
```

-- b) Add column Tot

```
ALTER TABLE Student
ADD Tot INT;
```

-- c) Calculate total marks

```
UPDATE Student
SET Tot=ENG+PHY+CHE+MATHS+CS;
```

-- d) Add columns Average and Grade

```
ALTER TABLE Student
ADD Average DECIMAL(5,2),
ADD Grade VARCHAR(2);
```

-- e) Calculate Average

```
UPDATE Student
SET Average=Tot/5;
```

```
UPDATE Student
SET Grade='A+'
WHERE Average>91;
```

```
-- f) Delete students whose Tot is not assigned
DELETE FROM Student
WHERE Tot IS NULL;
```

```
-- g) Display students whose Average is more than 80
SELECT *
FROM Student
WHERE Average>80;
```

```
-- h) Display minimum, maximum, sum and average of CS marks
SELECT MIN(CS), MAX(CS), SUM(CS), AVG(CS)
FROM Student;
```

```
-- i) Display ID, Name and marks in descending order of Average
SELECT id, name, ENG, PHY, CHE, MATHS, CS
FROM Student
ORDER BY Average DESC;
```

```
-- j) Add the record
INSERT INTO Student(id,name,ENG,PHY,CHE,MATHS,CS)
VALUES('s104','Catherine',95,76,NULL,74,63);
```

```
-- k) Change CHE to 80 and Grade to A for s104
UPDATE Student
SET CHE=80, Grade='A'
WHERE id='s104';
```

Exp No: 33

Date: dd-mm-yyyy

Aim: Consider Tables **EMP** and **Dept**

empno	empname	job	mgr	hiredate	sal	comm	deptno
7369	Smith	Clerk	7902	1980-12-17	800	NULL	NULL
7499	Allen	Salesman	7698	1981-02-20	1600	300	30
7521	Ward	Salesman	7698	1982-02-22	1250	500	30
7566	Jones	Manager	7839	1981-04-02	2975	NULL	20
7654	Martin	Salesman	7698	1981-09-28	1250	1400	30
7698	Blake	Manager	7839	1981-05-01	2850	NULL	30
7782	Clark	Manager	7839	1981-06-09	2450	NULL	10
7788	Scott	Analyst	7566	1982-12-09	3000	NULL	20
7839	King	President	NULL	1981-11-17	5000	NULL	10
7844	Turner	Salesman	7698	1981-09-08	1500	0	30
7876	Adams	Clerk	7788	1983-01-23	1100	NULL	20
7900	James	Clerk	7698	1981-12-03	950	NULL	30
7902	Ford	Analyst	7566	1981-12-03	3000	NULL	NULL
7934	Miller	Clerk	7782	1982-01-23	1300	NULL	NULL

deptno	dname	loc
10	Accounting	New york
20	Research	Dallas
30	Sales	Chicago
40	HR	Boston
50	Operations	Bangalore

- a) Increase the salary of all employees by a factor of 100.
- b) Reduces salaries by 100 for employees whose salary is above the average salary, but never below 500, in one transaction.
- c) Display number of employees, sum and average salary in each department from tables Emp and Dept.
- d) Display Empno, Empname, and Deptno & Dname from tables Emp and Dept.
- e) Display number of employees, sum and average salary in each job.
- f) Display number of employees, sum and average salary in each job where number of employees less than 2.
- g) Display the name and joining date of senior most employee.
- h) Display minimum salary and last hiring date.
- i) Display average commission from the table Employee.
- j) Display different jobs.
- k) Display number of different jobs.

```
CREATE TABLE Dept
```

```
(
  deptno INT PRIMARY KEY,
  dname VARCHAR(20),
  loc VARCHAR(20)
);
```

```
CREATE TABLE EMP
```

```
(
  empno INT PRIMARY KEY,
  empname VARCHAR(20),
  job VARCHAR(20),
  mgr INT,
  hiredate DATE,
  sal INT,
  comm INT,
  deptno INT,
  FOREIGN KEY (deptno) REFERENCES Dept(deptno)
);
```

```
INSERT INTO Dept VALUES
```

```
(10, 'Accounting', 'New york'),
(20, 'Research', 'Dallas'),
(30, 'Sales', 'Chicago'),
(40, 'HR', 'Boston'),
(50, 'Operations', 'Bangalore');
```

```
INSERT INTO EMP VALUES
```

```
(7369, 'Smith', 'Clerk', 7902, '1980-12-17', 800, NULL, NULL),
(7499, 'Allen', 'Salesman', 7698, '1981-02-20', 1600, 300, 30),
(7521, 'Ward', 'Salesman', 7698, '1982-02-22', 1250, 500, 30),
(7566, 'Jones', 'Manager', 7839, '1981-04-02', 2975, NULL, 20),
(7654, 'Martin', 'Salesman', 7698, '1981-09-28', 1250, 1400, 30),
(7698, 'Blake', 'Manager', 7839, '1981-05-01', 2850, NULL, 30),
(7782, 'Clark', 'Manager', 7839, '1981-06-09', 2450, NULL, 10),
(7788, 'Scott', 'Analyst', 7566, '1982-12-09', 3000, NULL, 20),
```

```
(7839, 'King', 'President', NULL, '1981-11-17', 5000, NULL, 10),
(7844, 'Turner', 'Salesman', 7698, '1981-09-08', 1500, 0, 30),
(7876, 'Adams', 'Clerk', 7788, '1983-01-23', 1100, NULL, 20),
(7900, 'James', 'Clerk', 7698, '1981-12-03', 950, NULL, 30),
(7902, 'Ford', 'Analyst', 7566, '1981-12-03', 3000, NULL, NULL),
(7934, 'Miller', 'Clerk', 7782, '1982-01-23', 1300, NULL, NULL);
```

-- a)

```
UPDATE EMP
```

```
SET sal = sal + 100;
```

-- b)

```
START TRANSACTION;
```

```
UPDATE EMP
```

```
SET sal = GREATEST(sal - 100, 500)
```

```
WHERE sal >
```

```
  (SELECT avgsal
```

```
    FROM (SELECT AVG(sal) AS avgsal FROM EMP) A);
```

```
COMMIT;
```

-- c)

```
SELECT D.deptno, D.dname, COUNT(E.empno), SUM(E.sal), AVG(E.sal)
```

```
FROM EMP E, Dept D
```

```
WHERE E.deptno = D.deptno
```

```
GROUP BY D.deptno, D.dname;
```

-- d)

```
SELECT E.empno, E.empname, E.deptno, D.dname
```

```
FROM EMP E, Dept D
```

```
WHERE E.deptno = D.deptno;
```

-- e)

```
SELECT job, COUNT(empno), SUM(sal), AVG(sal)
```

```
FROM EMP
```

```
GROUP BY job;
```

-- f)

```
SELECT job, COUNT(empno), SUM(sal), AVG(sal)
```

```
FROM EMP
```

```
GROUP BY job
```

```
HAVING COUNT(empno) < 2;
```

-- g)

```
SELECT empname, hiredate
```

```
FROM EMP
```

```
WHERE hiredate = (SELECT MIN(hiredate) FROM EMP);
```

-- h)

```
SELECT MIN(sal), MAX(hiredate)
```

```
FROM EMP;
```

```
-- i)
SELECT AVG(comm)
FROM EMP;

-- j)
SELECT DISTINCT job
FROM EMP;

-- k)
SELECT COUNT(DISTINCT job)
FROM EMP;
```

Exp No: 34

Date: dd-mm-yyyy

Aim: Consider Tables: Flights & Fares

FLNO	Start	End	NoFlights	NoStops
Am812	Delhi	Mumbai	8	0
IC301	Mumbai	Delhi	8	0
IC799	Bangalore	Delhi	2	1
IC899	Kanpur	Bangalore	1	1
MC101	Indore	Mumbai	3	0

FLNO	Airlines	Fare	Tax
AM501	JetAirways	13450	8
AM812	Indian Airlines	10500	10
IC301	Indian Airlines	5200	10
IC302	Indian Airlines	4300	10
IC701	Indian Airlines	6200	10
IC899	Indian Airlines	8300	10
MC101	Deccan Airlines	3500	4
MU499	Sahara	9400	5

Write commands to display:

- FI_No and No_Flights from Kanpur to Bangalore.
- Arrange the contents of flights in descending order of FI_No.
- Display FI_No, Airlines and Fare to be paid for the flights from Delhi to Mumbai from tables flights and Fares where Fare to be paid = Fare + Fare * Tax / 100.
- Minimum fare Indian Airlines offering from table Fares.

CREATE TABLE Flights

```
(
  FLNO VARCHAR(5) PRIMARY KEY,
  `Start` VARCHAR(20),
  `End` VARCHAR(20),
  NoFlights INT,
  NoStops INT);
```

CREATE TABLE Fares

```
(
  FLNO VARCHAR(5) PRIMARY KEY,
  Airlines VARCHAR(30),
  Fare INT, Tax INT);
```

```

INSERT INTO Flights VALUES
('AM812','Delhi','Mumbai',8,0),
('IC301','Mumbai','Delhi',8,0),
('IC799','Bangalore','Delhi',2,1),
('IC899','Kanpur','Bangalore',1,1),
('MC101','Indore','Mumbai',3,0);

```

```

INSERT INTO Fares VALUES
('AM501','JetAirways',13450,8),
('AM812','Indian Airlines',10500,10),
('IC301','Indian Airlines',5200,10),
('IC302','Indian Airlines',4300,10),
('IC701','Indian Airlines',6200,10),
('IC899','Indian Airlines',8300,10),
('MC101','Deccan Airlines',3500,4),
('MU499','Sahara',9400,5);

```

```

-- a)
SELECT FLNO, NoFlights
FROM Flights
WHERE `Start`='Kanpur' AND `End`='Bangalore';

```

```

-- b)
SELECT *
FROM Flights
ORDER BY FLNO DESC;

```

```

-- c)
SELECT F.FLNO, R.Airlines,
       R.Fare + R.Fare * R.Tax / 100 AS Fare_To_Be_Paid
FROM Flights F, Fares R
WHERE F.FLNO = R.FLNO
AND F.`Start`='Delhi'
AND F.`End`='Mumbai';

```

```

-- d)
SELECT MIN(Fare)
FROM Fares
WHERE Airlines='Indian Airlines';

```

Aim: Consider Tables : Employee & Department.

Table : Employee

EMPID	Name	DEPT	Gender	Experience	SALARY
EM001	MR. HARI	P10	M	20	100000
EM002	MS. SREYA	P11	F	17	75000
EM003	MR. NIRMAL	P12	M	18	150000
EM004	MS. RENU	P12	F	14	112000
EM005	MS. TEENA	P13	F	21	90000
EM006	MS. PRIYA	P10	F	19	200000

Table: Department

D_ID	D_Name	INCENTIVE
P10	FINANCE	10
P11	HR	8
P12	IT	15
P13	ADMIN	12

- Display the names of employees and the name of the department they are in.
- Display the number of male and female employees in each department.
- Display the number of employees having more than 15 yrs of experience and earning more than 100000.
- Display the name of the employees and the incentive that they will get . Incentive column has the values in percentage. The values that should be displayed is the calculated value of the percentage on the salary.
- Display the total salary of each employee along with the name of the employee which is salary + incentive(calculate percentage on the salary)

CREATE TABLE Department

```
( D_ID VARCHAR(3) PRIMARY KEY,
  D_Name VARCHAR(20),
  INCENTIVE INT);
```

CREATE TABLE Employee(

```
EMPID VARCHAR(5) PRIMARY KEY,
Name VARCHAR(20),
DEPT VARCHAR(3),
Gender CHAR(1),
Experience INT,
SALARY INT,
FOREIGN KEY (DEPT) REFERENCES Department(D_ID));
```

INSERT INTO Department VALUES

```
('P10','FINANCE',10),('P11','HR',8),('P12','IT',15),('P13','ADMIN',12);
```

INSERT INTO Employee VALUES

```
('EM001','MR. HARI','P10','M',20,100000),
('EM002','MS. SREYA','P11','F',17,75000),
('EM003','MR. NIRMAL','P12','M',18,150000),
('EM004','MS. RENU','P12','F',14,112000),
('EM005','MS. TEENA','P13','F',21,90000),
('EM006','MS. PRIYA','P10','F',19,200000);
```

-- a)

```
SELECT E.Name, D.D_Name  
FROM Employee E, Department D  
WHERE E.DEPT = D.D_ID;
```

-- b)

```
SELECT DEPT, Gender, COUNT(*)  
FROM Employee  
GROUP BY DEPT, Gender;
```

-- c)

```
SELECT COUNT(*)  
FROM Employee  
WHERE Experience > 15  
AND SALARY > 100000;
```

-- d)

```
SELECT E.Name,  
       E.SALARY * D.INCENTIVE / 100 AS Incentive  
FROM Employee E, Department D  
WHERE E.DEPT = D.D_ID;
```

-- e)

```
SELECT E.Name,  
       E.SALARY + (E.SALARY * D.INCENTIVE / 100) AS Total_Salary  
FROM Employee E, Department D  
WHERE E.DEPT = D.D_ID;
```